

Lab experience 5: Accelerometer-based CPR feedback device

LabVIEW Version

Objectives

- Gain experience using an electronic accelerometer
- Practice using LabVIEW's integration and filtering functions
- Use LabVIEW and the accelerometer to create a CPR chest compression monitor.

Reading & new concepts

- SubVI generation (covered in the textbook)
- Peak detection

Hardware

As you might guess, an accelerometer measures acceleration, including the virtual acceleration produced by Earth's gravitational field. For this week's activities we are using an Analog Devices ADXL335 3-axis accelerometer installed on a printed circuit board with a voltage regulator and appropriate capacitors and connectors.¹ The ADXL335 has $\pm 3g$ range; which should be plenty for the CPR project. This accelerometer uses a surface-micromachined capacitive sensing cell as described in its datasheet.²

In order to use the accelerometer board, we need to supply power at the Vin and GND pins, and measure from the X, Y and/or Z out pins. Zout is the most important for its use for CPR feedback; use of Xout and Yout is optional. The other pins may remain disconnected.



A header (row of pins) has been soldered to each board, and you will be given long jumper wires to allow you to connect the DAQ to the accelerometer board. The power input at Vin may be supplied by the 5 V supply on the DAQ, so the minimal setup consists only of the DAQ, accelerometers and connectors.

¹ <https://www.adafruit.com/product/163>

² <https://www.analog.com/media/en/technical-documentation/data-sheets/ADXL335.pdf>

Software Overview (Instructions in Programming)

We will use LabVIEW to create a VI that performs data acquisition and signal processing functions, and provides a user interface.

Data acquisition: You may collect 1, 2, or 3 channels, which means you may need multiple DAQ assistants. Using single-ended mode (**RSE**) simplifies the connections, because the X, Y, Z outputs and Vin use a common reference. All of the voltages are in the **range 0 – 5 V**. Process each sample as it comes in, so **selecting one channel that takes 1 Sample (On Demand)** is recommended.

Signal processing: The X, Y, Z signals are referenced to some positive voltage, not to 0 V. Therefore, it is wise to zero the acquired input so the X, Y, or Z signal *is* referenced to 0. This requires you to save the measured voltage in one loop and subtract it from all future measurements. I use a Case structure that allows me to click on a button when I want to capture that reference voltage. In addition, you will need to convert the voltage to SI units. The ADXL335 data sheet provides a mV-to-*g* sensitivity, and you can probably figure out the next step in the conversion.

Velocity is the integral of the acceleration over time. LabVIEW has two integration functions, one for arrays and one Point-by-Point function. The array option is not usable here because you are capturing only one sample per channel. Like a Waveform Chart, the Point-by-Point function includes a buffer to accumulate the samples across loop iterations. However, I found it difficult to zero the velocity with the point-by-point function, so I wrote my own integrator using a shift register and multiplying by the time increment. Again, I used a case structure to select when to set the velocity to zero; this time you would use a zero scalar (for one DAQ channel) or array constant (for multiple channels) rather than subtracting an offset voltage.

Position is, of course, the integral of velocity over time. Again, I wrote an integrator and zeroing function into the VI. A new problem, however, is drift over time. Inexpensive accelerometers such as ours do not stay at exactly the same output voltage when sitting still, and after two integrations this error becomes intractably large. In the case of a CPR monitor, however, we are lucky because we know the device is not going to deviate from a small position range, and we are interested in the oscillatory motion rather than absolute position. Therefore, we can use a high-pass filter to eliminate the positional drift over time. For this purpose, I recommend a Point-by-Point FIR filter: Signal Processing >> Point-by-Point >> Filters. The point-by-point functions take in one scalar every loop, just like a waveform chart. There are also filters that take an array as input, just like a waveform graph, but you don't want that kind here. You will need one filter per variable (velocity and position). Notes: (1) For a low or high pass filter, you only need the Low Cutoff Frequency. (2) Remember to wire up the sampling frequency, which you will calculate from the loop wait time.

Pre-lab calculations

1. Verify from the device specifications that the DAQ's 5 V terminal can supply adequate voltage and current for the accelerometer board.
2. Determine the theoretical voltage-to-acceleration conversion in SI units. (*Hint:* For the conversion factor, look for the typical sensitivity in the ADXL335 datasheet)
3. Determine the predicted maximum acceleration if the CPR rate is 120 compressions per minute and the compression depth is 50 mm. Assuming CPR motion is sinusoidal.

Programming

1. Start your VI with a while loop with the usual Stop control and msec timer. Add the ADXL335Simulator.vi subVI within the loop. This subVI produces a three-element 1D array representing the X, Y and Z voltages from the ADXL335 board. The subVI assumes that the accelerometer is horizontal, i.e. the Z direction is pointed opposite to gravity. Wire the loop counter **i**, msec timer value, and a control for whether CPR is being administered. When the CPR Boolean is set true, the third element in the array output is a 1-Hz sinusoid; when the Boolean is false, the output is a drifting constant plus noise. The X and Y outputs are always a constant plus noise.
2. Add a Waveform Chart to the front panel and use it to display the X, Y and Z signals with and without CPR. Remember to insert Build Array to display the 3 voltages if necessary. Notice that when you turn on CPR, the third (Z) channel will be sinusoidal, while the other two (X and Y) remain at the same levels.

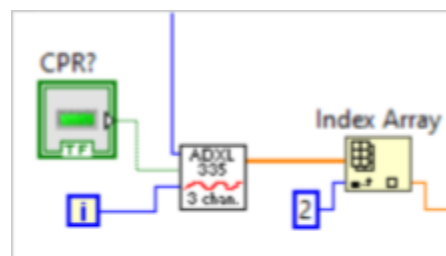


Figure 1. Extracting Z signal from simulator subVI.

3. From now on, you only need to work with the Z signal. Extract the Z signal using “Index Array” (Figure 1).

Note: Starting from this step, there is a lot of freedom in designing your VI. You are welcome to experiment with different functions and parameters.

4. Notice that the Z-signal is non-zero even when CPR is turned off. We must subtract this offset voltage from the Z-signal because this offset is not due to the actual CPR movement. Write code to remove this offset. There are many ways of doing this.

Our solution: Create a shift register and a case structure. Create a control for the case structure, and name this boolean control “Zero”. When the case the true:

- Store the Z-signal to the shift register. Better yet, instead of just storing one sample of Z-signal, store the mean of ten samples using the Point by Point mean function (Go to Signal Processing > Point by Point > Prob & Stat > Mean).

When the case is false:

- Maintain the value of the stored in the shift register, i.e., passing the shift register value straight through the case structure without modifying it.
- Subtract the stored value in the shift register (i.e., the offset) from the Z-signal to get the offset-free Z-signal.

5. Add a conversion from volts to g’s to acceleration in m/s^2 . See Pre-lab calculations, question 2.

6. Add shift registers and the functions necessary to integrate acceleration to velocity and to position with the backward Euler method:

$$v_n = v_{n-1} + \Delta t \times a_n$$

where a_n is the offset-free acceleration, and v_{n-1} is the velocity that was stored in the shift register during the last loop iteration (Figure 3).

Hint: You probably need two more shift registers (for Δt)

Note: The Euler method is particularly unstable when your loop time Δt is large, but errors may accumulate when there are too many steps/iterations. Choose your loop time wisely.. Also, since the ADXL335Simulator.vi is set to output a 1-Hz sinusoid, make sure your loop time Δt is well below the Nyquist limit of 500 ms.

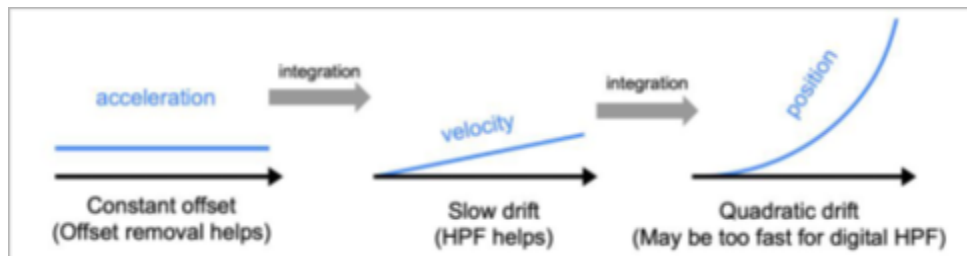


Figure 2. Diagram showing why it is difficult to remove drift from the position signal. Acceleration signal (**left**) comes with a small yet non-zero offset even after offset subtraction (offset measurements are hardly exact). The integration of this supposedly offset-free acceleration results in a slow, linear drift in the velocity signal (**middle**). A high pass filter (HPF) should remove this noise to some degree. However, the integration of this drift could easily result in a quadratic drift in the position signal (**right**). It is a matter of time before the position becomes intractably large.

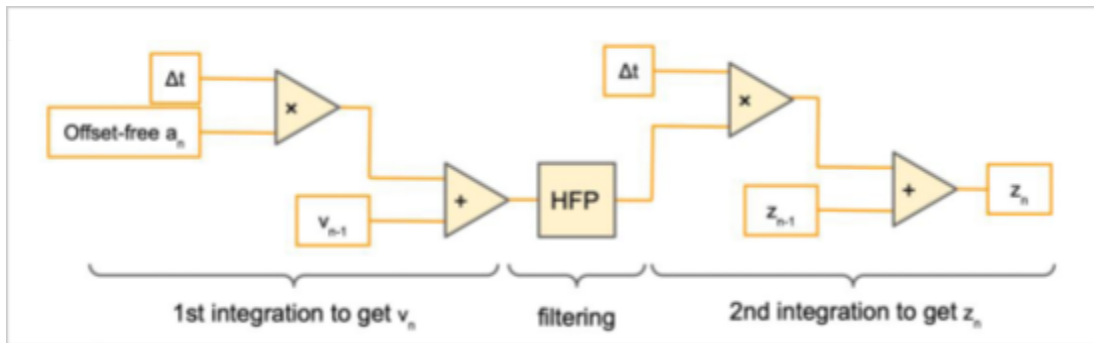


Figure 3. Recommended integration scheme to achieve drift-free position (z_n).

7. As shown in Figure 3, add a **point-by-point Butterworth high-pass filter** before the 2nd integration (Go to Signal Processing > Point by Point > Filters > Butterworth).



(which looks like this!)

This helps remove the drift in velocity. Configure the filter as follows:

- Create a constant for the **filter type**. Set it to Highpass.
- Create a constant for the **low cutoff frequency**. Set it to 0.5 Hz for now.
- Create a constant for the **order**. Set it to 6.
- Connect the “Zero” Boolean wire to the **initialize** terminal on the top of the filter icon. This resets the internal buffer of the point-by-point filter without stopping the VI.
- Use two **High Resolution Relative Seconds** VIs to measure the loop time (you did this in Lab 1). Then take the inverse to get the sampling rate. Connect it to the sampling rate terminal of the filter.

Note: Do not pass the filtered velocity and position into the shift registers. (It is okay if the position is calculated from the filtered velocity, though).

8. Create a waveform plot to visualize the filtered velocity.
9. Now we want to visualize the filtered position: Create another filter with the same configuration above. Next, pass the raw position to this filter, then pass the filtered position to a waveform plot.
10. We would also like to zero out the velocity and position when needed. Write code such that when the boolean control “Zero” is true, the velocity and position stored inside the shift registers are zeroed.

The following steps might take some in-person help, and may be completed in lab.

11. Incorporate a point-by-point peak detector that allows you to measure the peak-to-peak interval (and hence CPR rate) as well as the CPR amplitude. To find the amplitude, you will have to configure the peak detector as a trough detector and

subtract successive peak and trough values (Figure 4). If your position signal contains a lot of drift, you may still use the acceleration signal to determine the CPR rate.

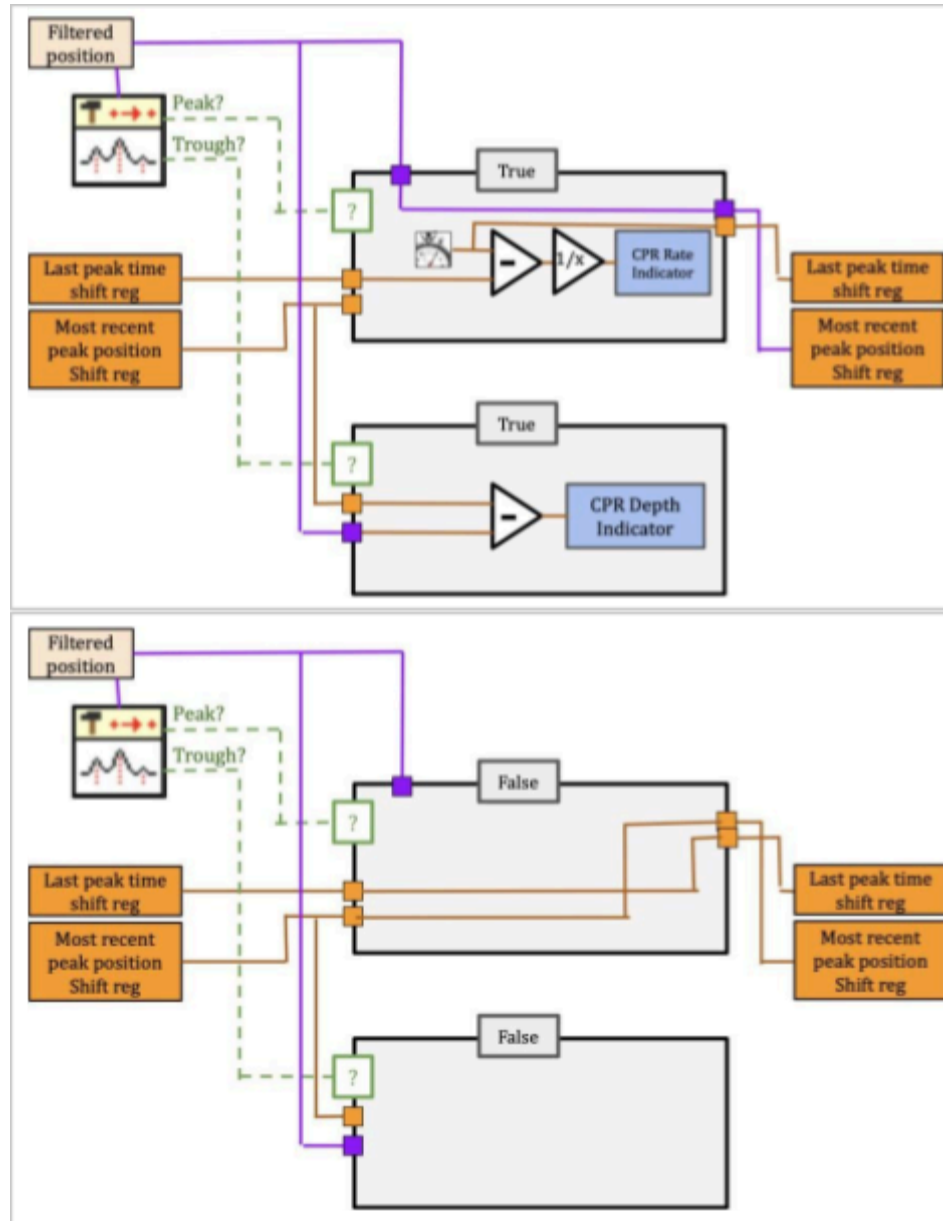


Figure 4. One possible way of calculating CPR rate and amplitude with a point-by-point peak detector. The upper figure shows the case when case structures are true, the lower one shows the case when the structures are false.

Optional: Incorporate a Schmitt trigger to convert the CPR waveform into a square wave to determine CPR rate. Note that this method does not provide CPR amplitude.

12. Add user feedback to the VI's front panel. The VI should show the rate and depth, and indicate whether CPR is being performed according to AHA's CPR standard (see the link on the next page). The user should be able to set depth and rate thresholds. Audible feedback is optional.

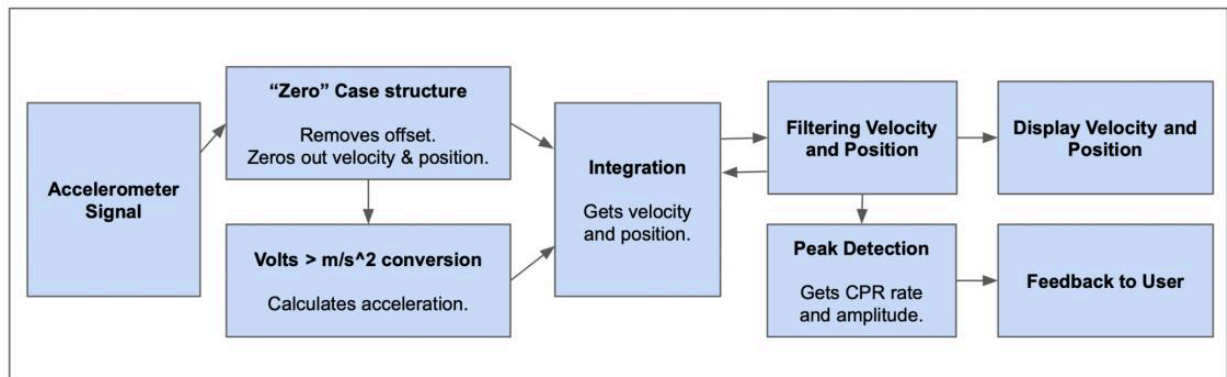
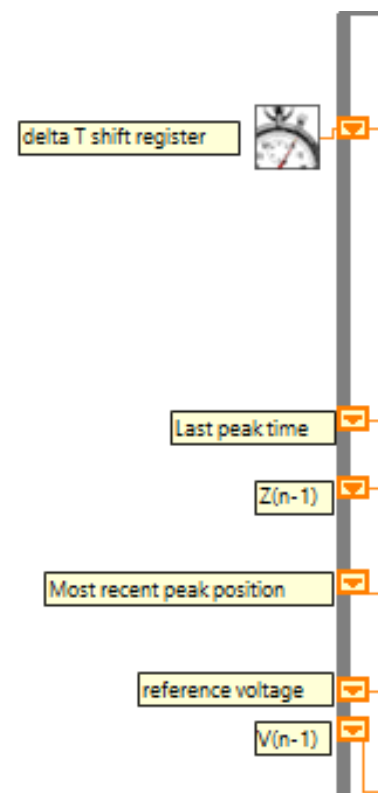


Figure 5. An example of the functional partitioning of the VI. Yours may look different.



Here are the shift registers necessary to create this VI:

In-Lab Activities

1. Obtain the ADXL335 accelerometer board and a NI USB-6001 DAQ module.
2. Open your pre-lab VI and replace the simulated accelerometer with the DAQmx assistant. Suggested settings are described above in the Software section.
3. Apply sinusoidal, CPR-like movement to your accelerometer and observe the results. Revise your VI as necessary.
4. Collect data and report the results in your [write-up](#). A table is provided.
 - 60 bpm and 120 bpm for CPR rate: we recommend using a metronome app.
 - 5 cm and 15 cm for CPR depth: obtain a ruler from the instructor.
5. Demonstrate your working VI to an instructor.

Optional Activities

Add feedback to notify the user if their CPR rate or depth is out of the ideal range

Obtain/create an outer case for your accelerometer unit, such that it could be used between a resuscitator's hands and a resuscitee's sternum during CPR. This could be part of a final class project.

Deliverables

1. Your final VI.
2. A [write-up](#)